

Lab #2. Modules – Part of HW #2

2024-05-18

This assignment will teach you how to use modules and packages and where they can be used. Put all your Colab code in a single notebook for easier submission.

Contents

Introduction to Modules.....	1
1. os and request Modules:.....	2
Task 1 Submission. Provide a small paragraph summary of your discoveries in Task 1 and add screenshots of your code and results from all 3 environments (no need to attach any code)	3
2. Walking through directories.....	3
Coding Challenge: Connect to obi2, Create a script that navigates through all subdirectories in a public_html directory and lists all files with a specific extension (e.g., .py). You can choose the script location yourself	6
Task 2 Submission. Submit all source code and output from Task 2.....	6
3. Other modules frequently in use – Better on Colab	6
Task 3. Make sure all code runs, all output clearly seen, all graphs are there if any. You will print all Google Colab code at once.....	10
4. requests Module	10
run the app	14
Task 4 Submission. Submit all source code and output from Task 3 including browser screenshots if any.....	14
5. flask Module – use VSC.....	14
Task 5 Submission. Submit all source code and output from Task 4, including browser screenshots if any.....	18
6. Use Google Colab and unittest to Create your own module and package	18
7. Lab Submission	19

Introduction to Modules

Modules in Python are essential for organizing and structuring code efficiently. They allow developers to break down large programs into smaller, manageable, and reusable pieces. By encapsulating related functions, classes, and variables into separate files, modules promote code reusability, readability, and

maintainability. Using modules enables the **separation of concerns**, allowing different parts of an application to be developed and maintained independently.

Let's try out modules on obi2.

1. os and request Modules:

os Module offers functions for interacting with the OS (e.g., getting the home directory).

1.1. Create a new folder in your computer called Lab2_yourname

1.2. Create a new python file system_check.py. The main content is below.

Analyze the script first (read through it and try to understand), watch out for bad indentation.

```
def gather_system_info():
    system_info = {
        "os": platform.system(),
        "os_version": platform.version(),
        "architecture": platform.architecture(),
        "machine": platform.machine(),
    }

    if system_info["os"] == "Linux":
        system_info["cpu_info"] = subprocess.check_output("lscpu", shell=True).decode()
        system_info["memory_info"] = subprocess.check_output("free -h", shell=True).decode()
        system_info["disk_info"] = subprocess.check_output("df -h", shell=True).decode()
    elif system_info["os"] == "Windows":
        system_info["cpu_info"] = subprocess.check_output("wmic cpu get caption, deviceid, name,
numberofcores, maxclockspeed, status", shell=True).decode()
        system_info["memory_info"] = subprocess.check_output("wmic memorychip get capacity, speed",
shell=True).decode()
        system_info["disk_info"] = subprocess.check_output("wmic logicaldisk get size,freespace,caption",
shell=True).decode()
        system_info["system_info"] = subprocess.check_output("systeminfo", shell=True).decode()

    return system_info

def save_system_info(info):
    with open("system_info.json", "w") as f:
        json.dump(info, f, indent=4)

if __name__ == "__main__":
    system_info = #complete this code
    save_system_info(#complete this code)
    print("System information saved to system_info.json")
```

1.3. Import required modules

```
import platform
import subprocess
import json
```

1.4. Complete the missing code and run the program.

1.5. Find out where your output is. Screenshot your trial and its output.

And save it all for further submission.

1.6. Run the same code on Google colab, debug if needed.

Find your output, take a screenshot of your code and output.

1.7. Connect to obi2 and run your code there as well. Screenshot your results and the output.

Task 1 Submission. Provide a small paragraph summary of your discoveries in Task 1 and add screenshots of your code and results from all 3 environments (no need to attach any code)

2. Walking through directories

2.1. Linux commands exploration

Connect to obi2 unless you are already there.

As you recall, in lab1 we create files and directories with permissions.

Let's create 1 more manually.

```
Last login: Thu May 23 13:17:39 2024 from 10.1
ykumar@kean.edu@obi2 ~]$ cd public_html
ykumar@kean.edu@obi2 public_html]$ mkdir Lab2
ykumar@kean.edu@obi2 public_html]$ cd Lab2
ykumar@kean.edu@obi2 Lab2]$
```

We can then create a file **test.py** to print your name and test it. Complete this step on your own

```
ykumar@kean.edu@obi2 Lab2]$ vi test.py
ykumar@kean.edu@obi2 Lab2]$ python test.py
YK
ykumar@kean.edu@obi2 Lab2]$
```

Command **pwd** can show you the path

Command **ls** can list files and directories in your current directory

chmod command can change permissions to your file

Test them out

```
ykumar@kean.edu@obi2 Lab2]$ ls
test.py
ykumar@kean.edu@obi2 Lab2]$ pwd
/home/student/ykumar/public_html/Lab2
ykumar@kean.edu@obi2 Lab2]$ ls -la
total 8
drwxrwx---+ 2 ykumar@kean.edu student 21 May 26 11:53 .
drwxr-xr-x+ 10 ykumar@kean.edu student 4096 May 26 11:47 ..
-rw-rw----+ 1 ykumar@kean.edu student 12 May 26 11:53 test.py
ykumar@kean.edu@obi2 Lab2]$ chmod 755 test.py
ykumar@kean.edu@obi2 Lab2]$ ls -la
total 8
drwxrwx---+ 2 ykumar@kean.edu student 21 May 26 11:53 .
drwxr-xr-x+ 10 ykumar@kean.edu student 4096 May 26 11:47 ..
-rwxr-xr-x+ 1 ykumar@kean.edu student 12 May 26 11:53 test.py
ykumar@kean.edu@obi2 Lab2]$
```

Take **screenshot** of your trials

2.2. Do it programmatically.

Create a file `myOS.py` on obi2 and **analyze it first** and then try this code all within Lab2 directory.

```
import os
import platform

# List all files and directories
print("Current directory contents:", os.listdir('.'))

# Create a new directory
os.mkdir('test_dir')
print("Created 'test_dir':", 'test_dir' in os.listdir('.'))

# Rename a file
with open('old_name.txt', 'w') as f:
    f.write('Sample text')
os.rename('old_name.txt', 'new_name.txt')
print("Renamed file:", 'new_name.txt' in os.listdir('.'))

# Set and get file permissions (only for Unix-based systems)
if os.name == 'posix':
    os.chmod('new_name.txt', 0o644) # Set file permissions to rw-r--r--
    file_permissions = oct(os.stat('new_name.txt').st_mode)[-3:]
    print(f"File permissions for 'new_name.txt': {file_permissions}")
else:
    print("File permissions setting is not supported on this operating system")

# Additional OS-specific information
if platform.system() == "Windows":
    print("Running on Windows")
elif platform.system() == "Linux":
    print("Running on Linux")
elif platform.system() == "Darwin":
    print("Running on macOS")
else:
    print("Unknown operating system")
print("Script executed successfully.")
```

Debug if needed

The result should be similar to mine or better

```
Script executed successfully.
ykumar@kean.edu@obi2 Lab2]$ ls -la
total 16
drwxrwx---+ 3 ykumar@kean.edu student  72 May 26 12:42 .
drwxr-xr-x+ 10 ykumar@kean.edu student 4096 May 26 11:47 ..
-rw-rw---+ 1 ykumar@kean.edu student 1100 May 26 12:42 myOS.py
-rw-r--r---+ 1 ykumar@kean.edu student  11 May 26 12:42 new_name.txt
drwxrwx---+ 2 ykumar@kean.edu student   6 May 26 12:42 test_dir
-rwxr-xr-x+ 1 ykumar@kean.edu student  12 May 26 11:53 test.py
ykumar@kean.edu@obi2 Lab2]$
```

Take screenshot of your code and output

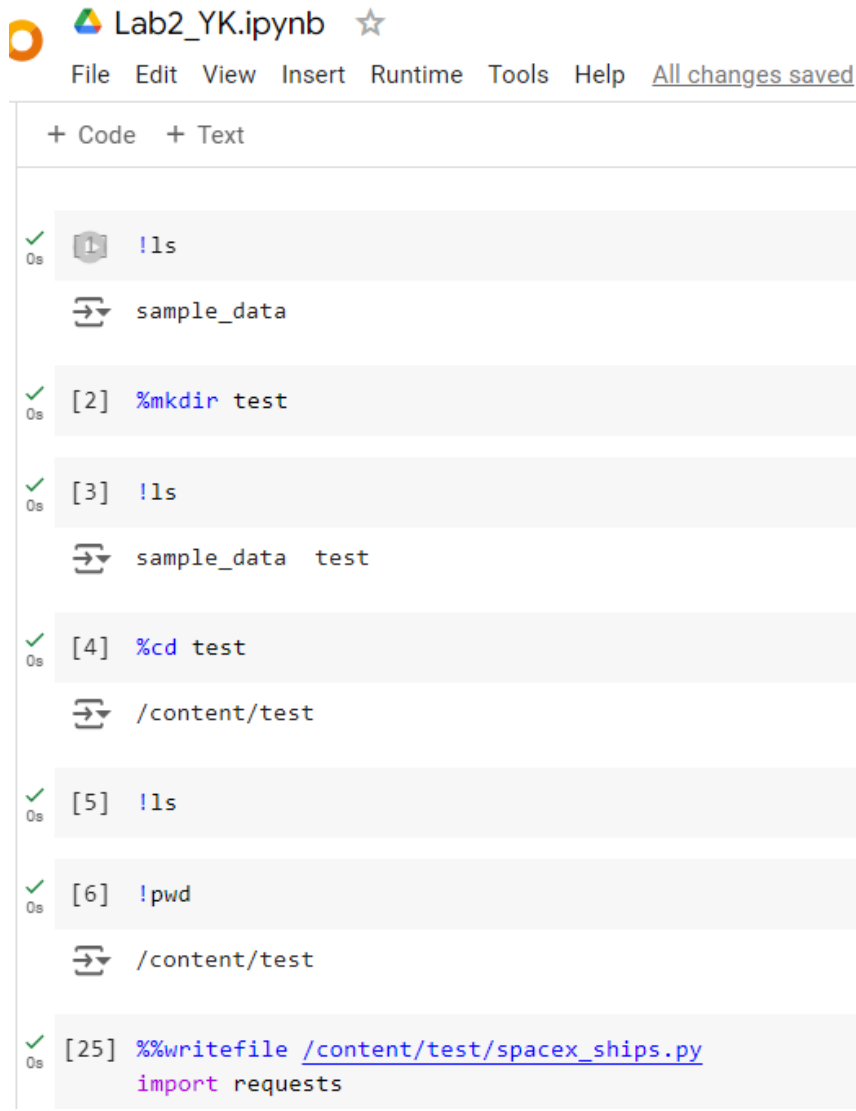
2.3. Try it on your machine, debug as needed

Take screenshot of your code and output

2.4. Travel through colab

Create a new Notebook, call is Lab2_**yourname**

Let's test the commands. Follow the script below



```
Lab2_YK.ipynb ☆
File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

✓ 0s [1] !ls
sample_data

✓ 0s [2] %mkdir test

✓ 0s [3] !ls
sample_data test

✓ 0s [4] %cd test
/content/test

✓ 0s [5] !ls

✓ 0s [6] !pwd
/content/test

✓ 0s [25] %%writefile /content/test/spacex_ships.py
import requests
```

text below

```
%%writefile /content/test/spacex_ships.py
import requests
import json

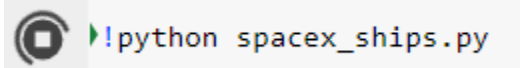
def fetch_data(endpoint):
    url = f"https://api.spacexdata.com/v3/{endpoint}"
    response = requests.get(url)
    if response.status_code == 200:
        return response.json()
    else:
        print(f"Failed to fetch data from {endpoint}")
        return []

def print_data(data):
```

```
print(json.dumps(data, indent=4))

if __name__ == "__main__":
    endpoint = input("Enter the SpaceX API endpoint (e.g., ships, launches, rockets): ")
    data = fetch_data(endpoint)
    print_data(data)
```

to run



Locate code on the left and take a screenshot of the file location, you will submit all Colab code at once

Coding Challenge: Connect to obi2, Create a script that navigates through all subdirectories in a public_html directory and lists all files with a specific extension (e.g., .py). You can choose the script location yourself

```
import os

def list_files_with_extension(root_dir, extension):
    for dirpath, dirnames, filenames in os.walk(root_dir):
        for filename in filenames:
            if filename.endswith(extension):
                print(os.path.join(dirpath, filename))

# Define the root directory and the file extension
root_directory = #complete this code
file_extension = #complete this code

# List all files with the specified extension
list_files_with_extension(#complete this code, #complete this code)
```

Task 2 Submission. Submit all source code and output from Task 2.

3. Other modules frequently in use – Better on Colab

In this part of the lab, we will get familiar with some other modules just to get used to, for each of the following below examples:

- 1) Run the code snippet.
- 2) Add a text block on top with the task number and state what it does. Chat with ChatGPT or other LLM to clarify if you are not sure and have questions about it.
- 3) Create an additional example of your choice below so you see both to test the same functionality. Make sure it works.
- 4) Add a comment on top of your example with your name or initial.
- 5) If possible do not take screen of every single snippet but rather print to PDF and save as a notebook when all is done. Make sure all output can be seen in the PDF.

Counter Object Creation: Initializes a Counter object with the string 'hello world', counting the frequency of each character.

```
[8] from collections import Counter
c = Counter('hello world')
print(c)
```

Counter({'l': 3, 'o': 2, 'h': 1, 'e': 1, ' ': 1, 'w': 1, 'r': 1, 'd': 1})

```
#added on 5/27 by YK
from collections import Counter
c = Counter('hello world')
print(c)

d = Counter('hello python')
print(d)
```

Counter({'l': 3, 'o': 2, 'h': 1, 'e': 1, ' ': 1, 'w': 1, 'r': 1, 'd': 1})
Counter({'h': 2, 'l': 2, 'o': 2, 'e': 1, ' ': 1, 'p': 1, 'y': 1, 't': 1, 'n': 1})

3.1. **datetime** Module

```
from datetime import datetime
print(datetime.now())
```

3.2. **re** Module:

```
import re
pattern = re.compile(r'\d+')
print(pattern.findall('There are 123 numbers in 456 this string.'))
```

3.3. **json** Module:

```
import json
data = '{"name": "John", "age": 30}'
parsed = json.loads(data)
print(parsed['name'])
```

3.4. **math** Module:

```
import math
print(math.pi)
print(math.sqrt(16))
```

3.5. **random** Module:

```
import random
print(random.randint(1, 100))
```

3.6. **itertools** Module

```
import itertools
for i in itertools.count(10):
    print(i)
    if i >= 15:
```

`break`

3.7. **os** Module:

```
import os
print(os.name)
print(os.listdir('.'))
```

3.8. **sys** Module:

```
import sys
print(sys.platform)
print(sys.version)
```

3.9. **collections** Module:

```
from collections import Counter
print(Counter('hello world'))
```

3.10. **functools** Module:

```
import functools
print(functools.reduce(lambda x, y: x + y, [1, 2, 3, 4, 5]))
```

3.11. **operator** Module:

```
import operator
print(operator.add(1, 2))
```

3.12. **subprocess** Module:

```
import subprocess
print(subprocess.run(['echo', 'Hello World!'], capture_output=True, text=True).stdout)
```

3.13. **csv** Module:

```
import csv
with open('example.csv', mode='w', newline='') as file:
    writer = csv.writer(file)
    writer.writerow(['name', 'age'])
    writer.writerow(['Alice', 30])
    writer.writerow(['Bob', 25])
with open('example.csv', mode='r') as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

3.14. **shutil** Module:

```
import shutil
shutil.copyfile('example.csv', 'example_copy.csv')
```

3.15. **pathlib** Module:

```
from pathlib import Path
```

```
path = Path('example.csv')
print(path.exists())
print(path.is_file())
```

3.16. **requests** Module:

```
import requests
response = requests.get('https://api.github.com')
print(response.status_code)
print(response.json())
```

3.17. **beautifulsoup4** Module

```
from bs4 import BeautifulSoup
html = '<html><head><title>Test</title></head><body><p>Hello, world!</p></body></html>'
soup = BeautifulSoup(html, 'html.parser')
print(soup.title.string)
print(soup.p.text)
```

3.18. **numpy** Module

```
import numpy as np
array = np.array([1, 2, 3, 4])
print(array * 2)
```

3.19. **pandas** Module

```
#add your name age can be fake
import pandas as pd
data = {'name': ['Alice', 'Bob', 'Charlie', 'YourName'], 'age': [30, 25, 35, 28]}
df = pd.DataFrame(data)
print(df.head())
print(df.describe())
print(df[df['age'] > 30])
df['height'] = [165, 180, 175, 170]
print(df)
```

3.20. **matplotlib** Module

#this code is continuation of 2.19

```
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 5))
plt.scatter(df['age'], df['height'], color='purple')
for i, name in enumerate(df['name']):
    plt.annotate(name, (df['age'][i], df['height'][i]), textcoords="offset points", xytext=(0,10), ha='center')
plt.xlabel('Age')
plt.ylabel('Height')
plt.title('Age vs Height')
plt.grid(True)
plt.show()
```

Task 3. Make sure all code runs, all output clearly seen, all graphs are there if any. You will print all Google Colab code at once at the end.

4. requests Module

4.1. As you already noticed in class with NASA's API the `requests` module allows you to send HTTP requests using Python.

Such a request returns a Response Object with all the response `data` (content, encoding, status, etc). There are many APIs we can try, for example, we can try <https://api.spacexdata.com/v3/launches>

See for more details: <https://docs.spacexdata.com/>

The screenshot shows the SpaceX API documentation on the right and a REST client interface on the left. The REST client interface has a sidebar with a tree view of API endpoints. The 'Launches' endpoint is selected and circled in red. The main area shows the 'GET All Launches' endpoint with the URL `https://api.spacexdata.com/v3/launches` also circled in red. Below the URL, there is a table of optional query strings.

Param	Sample
flight_id	5a9fc479ab70786ba5a:aaa
start/end	start=2017-06-22&end=2017-06-25
flight_number	60
launch_year	2018
launch_date_utc	2017-06-23T19:10:00Z
launch_date_local	2017-06-23T19:10:00Z
tbd	true
rocket_id	falconheavy

There are many other API.

Coding Challenge: Mount your Google Drive and create a folder in it manually. Navigate to this folder via commands on colab, find a path to it, via commands create a file and within that file connect to any API of your choice and display the response you get. You can try the something from SpaceX, NASA or those below, or any other

```
response = requests.get('https://api.github.com')
print(response.json())
```

```
response = requests.get('https://api.github.com/users/Microsoft')
```

```
data = response.json()
```

```
response = requests.get(f'https://api.nasa.gov/planetary/apod?api_key=DEMO_KEY')
```

```
data = response.json()
```

PS: Some API are also good but require keys: <https://openweathermap.org/api>, <https://www.alphavantage.co/>, <https://platform.openai.com/docs/overview>, <https://translation.googleapis.com/language/translate/v2>, etc

4.2. Here is another fun API

```
import requests
```

```
import json
```

```
from IPython.display import Image, display, HTML
```

```
# Fetch Mars Rover photos and save to a JSON file
```

```
def fetch_mars_rover_photos():
```

```
    url = 'https://api.nasa.gov/mars-photos/api/v1/rovers/curiosity/photos'
```

```
    params = {
```

```
        'sol': 1000,
```

```
        'api_key': 'DEMO_KEY',
```

```
        'page': 1,
```

```
        'per_page': 5
```

```
    }
```

```
    response = requests.get(url, params=params)
```

```
    return response.json()
```

```
photos = fetch_mars_rover_photos()
```

```
# Save the JSON response to a file
```

```
with open('mars_photos.json', 'w') as f:
```

```
    json.dump(photos, f, indent=2)
```

```
# Print the JSON response
```

```
print(json.dumps(photos, indent=2))
```

```
# Function to display photos from a JSON file
```

```
def display_photos_from_file(file_path):
```

```
    with open(file_path, 'r') as f:
```

```
        photos = json.load(f)
```

```
    html_content = ""
```

```
    for photo in photos['photos'][:5]: # Display first 5 photos for brevity
```

```
        html_content += f"""
```

```
        <div style='margin-bottom: 20px;'>
```

```
            <h3>Photo ID: {photo['id']}</h3>
```

```
            <p><b>Sol:</b> {photo['sol']}</p>
```

```
            <p><b>Camera:</b> {photo['camera']['full_name']} ({photo['camera']['name']})</p>
```

```
            <p><b>Earth Date:</b> {photo['earth_date']}</p>
```

```
            <p><b>Rover:</b> {photo['rover']['name']}</p>
```

```
            <p><b>Rover Status:</b> {photo['rover']['status']}</p>
```

```

        <p><b>Landing Date:</b> {photo['rover']]['landing_date']}</p>
        <p><b>Launch Date:</b> {photo['rover']]['launch_date']}</p>
        <img src='{photo['img_src']}' alt='Mars Rover Photo' style='width:200px;'>
    </div>
    <hr>
    """

```

```
display(HTML(html_content))
```

```

# Display photos from the saved JSON file
display_photos_from_file('mars_photos.json')

```

print your results to PDF

4.3. The APIs apps look best in the browser but can run locally on your machine.

Let's try that.

1. On VSC under folder lab2 create a folder **localapp**
2. Stand on it. Right-click -> open in integrated terminal
3. Install packages

pip install requests Pillow

or **pip3 install requests Pillow**

4.4. create a file mars.py in Lab2 folder, The text is below

```

import tkinter as tk
from tkinter import ttk, messagebox
from PIL import Image, ImageTk
import requests
from io import BytesIO

# Function to fetch Mars Rover photos
def fetch_mars_rover_photos(api_key, sol, camera):
    url = 'https://api.nasa.gov/mars-photos/api/v1/rovers/curiosity/photos'
    params = {
        'sol': sol,
        'camera': camera,
        'api_key': api_key
    }
    response = requests.get(url, params=params)
    return response.json()

# Function to display photos in the GUI
def display_photos(photos):
    for widget in photos_frame.winfo_children():
        widget.destroy()

    if 'photos' not in photos or len(photos['photos']) == 0:
        messagebox.showinfo("No Photos", "No photos found for the given sol and camera.")
        return

```

```

for photo in photos['photos']:
    img_url = photo['img_src']
    response = requests.get(img_url)
    img_data = response.content
    img = Image.open(BytesIO(img_data))
    img = img.resize((200, 200), Image.ANTIALIAS)
    img = ImageTk.PhotoImage(img)

    panel = tk.Label(photos_frame, image=img)
    panel.image = img
    panel.pack(side="left", padx=10, pady=10)

# Function to handle the fetch button click
def on_fetch_button_click():
    sol = sol_entry.get()
    camera = camera_combobox.get()
    if not sol:
        messagebox.showerror("Input Error", "Please enter the Martian sol.")
        return
    if not camera:
        messagebox.showerror("Input Error", "Please select a camera.")
        return

    photos = fetch_mars_rover_photos(api_key, sol, camera)
    display_photos(photos)

# API Key (Replace 'DEMO_KEY' with your actual NASA API key)
api_key = 'DEMO_KEY'

# Create the main window
root = tk.Tk()
root.title("Mars Rover Photos")

# Create and place the widgets
sol_label = tk.Label(root, text="Martian Sol (Day):")
sol_label.pack(pady=5)

sol_entry = tk.Entry(root)
sol_entry.pack(pady=5)

camera_label = tk.Label(root, text="Camera Type:")
camera_label.pack(pady=5)

camera_options = ['fhaz', 'navcam', 'mast', 'chemcam', 'mahli', 'mardi', 'rhaz']
camera_combobox = ttk.Combobox(root, values=camera_options)
camera_combobox.pack(pady=5)

fetch_button = tk.Button(root, text="Fetch Photos", command=on_fetch_button_click)
fetch_button.pack(pady=20)

photos_frame = tk.Frame(root)

```

```
photos_frame.pack(pady=10)
```

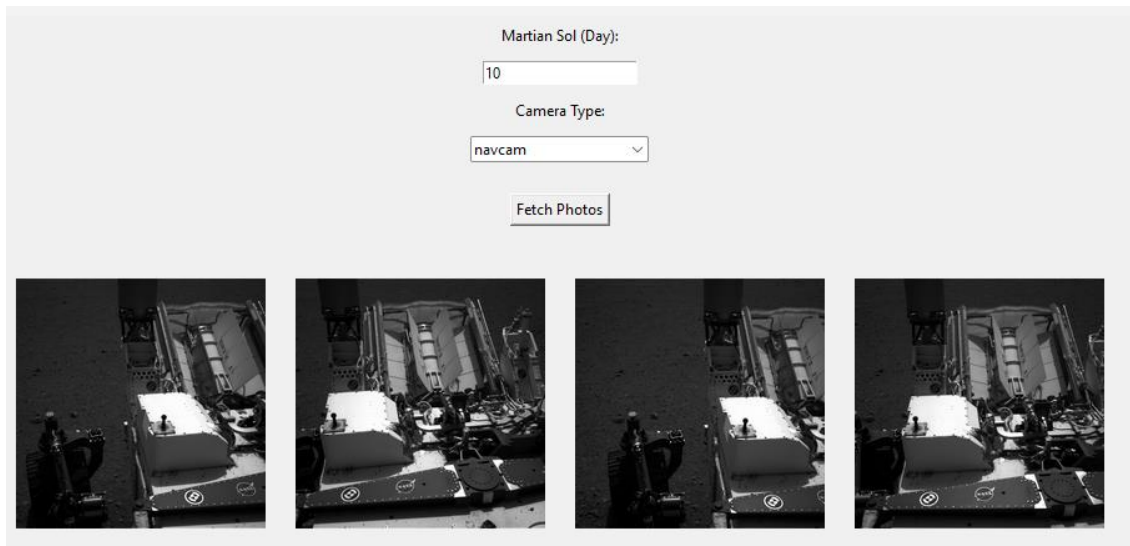
```
# Run the application  
root.mainloop()
```

run the app

python mars.py or python3 mars.py

4.5. Discuss with ChatGPT or a similar LLM all problems you encounter or code that you do not understand.

expected result is like this:



Task 4 Submission. Submit all source code and output from Task 4 including browser screenshots if any.

5. flask Module – use VSC

5.1. create a folder classed nasaapp for your project in VSC under Lab 2.

5.2. Install Flask

pip install flask requests Pillow
or pip3 install flask requests Pillow

5.3. create a file yourname_flask.py in this folder

```
from flask import Flask, render_template, request  
import requests  
from PIL import Image  
from io import BytesIO  
import base64  
  
app = Flask(__name__)
```

```

# Function to fetch Mars Rover photos
def fetch_mars_rover_photos(api_key, sol, camera):
    url = 'https://api.nasa.gov/mars-photos/api/v1/rovers/curiosity/photos'
    params = {
        'sol': sol,
        'camera': camera,
        'api_key': api_key
    }
    response = requests.get(url, params=params)
    return response.json()

# Route for the home page
@app.route('/', methods=['GET', 'POST'])
def home():
    photos = []
    if request.method == 'POST':
        sol = request.form.get('sol')
        camera = request.form.get('camera')
        api_key = 'DEMO_KEY' # Replace with your actual NASA API key

        photos_data = fetch_mars_rover_photos(api_key, sol, camera)

        if 'photos' in photos_data:
            for photo in photos_data['photos']:
                img_url = photo['img_src']
                response = requests.get(img_url)
                img_data = response.content
                img = Image.open(BytesIO(img_data))
                buffered = BytesIO()
                img.save(buffered, format="JPEG")
                img_str = base64.b64encode(buffered.getvalue()).decode()
                photos.append((photo['id'], photo['earth_date'], img_str))

    return render_template('index.html', photos=photos)

if __name__ == '__main__':
    app.run(debug=True)

```

5.4. Create the HTML Template index.html in the folder templates

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Mars Rover Photos</title>
    <style>
        .photo {
            margin: 20px;
            display: inline-block;

```

```

    }
    .photo img {
        width: 200px;
        height: 200px;
    }
</style>
</head>
<body>
    <h1>Mars Rover Photos</h1>
    <form method="POST">
        <label for="sol">Martian Sol (Day):</label>
        <input type="number" id="sol" name="sol" required>
        <br>
        <label for="camera">Camera Type:</label>
        <select id="camera" name="camera">
            <option value="fhaz">FHAZ</option>
            <option value="navcam">NAVCAM</option>
            <option value="mast">MAST</option>
            <option value="chemcam">CHEMCAM</option>
            <option value="mahli">MAHLI</option>
            <option value="mardi">MARDI</option>
            <option value="rhaz">RHAZ</option>
        </select>
        <br>
        <button type="submit">Fetch Photos</button>
    </form>
    <div>
        {% for photo in photos %}
        <div class="photo">
            <p>ID: {{ photo[0] }}</p>
            <p>Date: {{ photo[1] }}</p>
            
        </div>
        {% endfor %}
    </div>
</body>
</html>

```

5.5. Discuss with ChatGPT or a similar LLM all problems you encounter or code that you do not understand

5.6. Run the Flask Application:

```
python yourname_flask.py
```

5.7. Follow the link

```
PS C:\Users\galae\Desktop\00472-337513824 GS Backup 12.20.22 JC\KEAN_Lectures\33
20\Lab2\nasaapp> python yourname_flask.py
* Serving Flask app 'yourname_flask'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 106-583-628
```

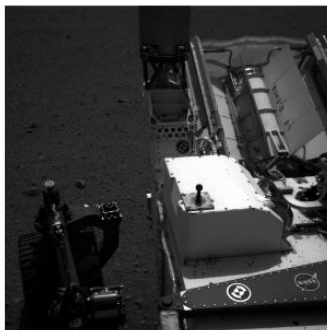
expected result is like this:

Mars Rover Photos

Martian Sol (Day):
Camera Type:

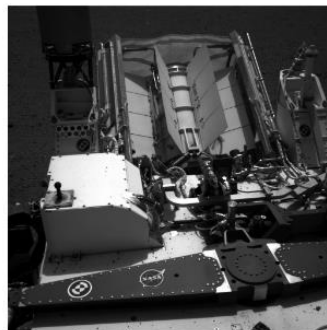
ID: 2095

Date: 2012-08-16



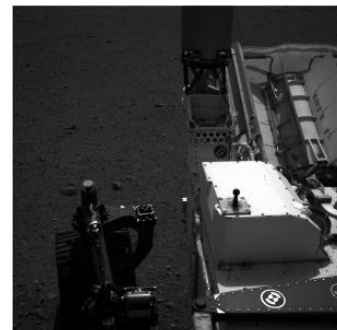
ID: 2672

Date: 2012-08-16



ID: 32406

Date: 2012-08-16



Task 5 Submission. Submit all source code and output from Task 5**6. Use Google Colab and `unittest` to Create your own module and package**

```
[84] !ls #see what is in the folder on your left
```

```
[74] # Create directories for the package
!mkdir -p myutils
!mkdir -p myutils/tests
```

```
[84] !ls #see what is in the folder on your left
```

```
[77] # Create the __init__.py file to make it a package
!touch myutils/__init__.py
!touch myutils/tests/__init__.py
```

```
[84] !ls #see what is in the folder on your left
```

```
# Write the module
%%writefile myutils/mathutils.py

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b
```

```
# Write the unit tests
%%writefile myutils/tests/test_mathutils.py
import unittest
from myutils.mathutils import add, subtract, multiply, divide

class TestMathUtils(unittest.TestCase):

    def test_add(self):
        self.assertEqual(add(2, 3), 5)
        self.assertEqual(add(-1, 1), 0)
        self.assertEqual(add(-1, -1), -2)

    def test_subtract(self):
        self.assertEqual(subtract(2, 3), -1)
        self.assertEqual(subtract(-1, 1), -2)
        self.assertEqual(subtract(-1, -1), 0)

    def test_multiply(self):
        self.assertEqual(multiply(2, 3), 6)
        self.assertEqual(multiply(-1, 1), -1)
        self.assertEqual(multiply(-1, -1), 1)

    def test_divide(self):
        self.assertEqual(divide(6, 3), 2)
        self.assertEqual(divide(-1, 1), -1)
        self.assertEqual(divide(-1, -1), 1)
        with self.assertRaises(ValueError):
            divide(1, 0)

if __name__ == '__main__':
    unittest.main(argv=[''], verbosity=2, exit=False)
```

```
[84] # Run the tests
!python -m unittest discover -s myutils/tests -p "test_*.py"
#mess up some math in your code like plus and minus swap making it not ok
#rerun, clue - if your made changes you need to rerun that write file
#COMPLETE THIS and record your FAILURE
```

The code is below

```
# Write the module
%%writefile myutils/mathutils.py

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
```

```

    raise ValueError("Cannot divide by zero")
return a / b

```

```

# Write the unit tests
%%writefile myutils/tests/test_mathutils.py
import unittest
from myutils.mathutils import add, subtract, multiply, divide

class TestMathUtils(unittest.TestCase):

    def test_add(self):
        self.assertEqual(add(2, 3), 5)
        self.assertEqual(add(-1, 1), 0)
        self.assertEqual(add(-1, -1), -2)

    def test_subtract(self):
        self.assertEqual(subtract(2, 3), -1)
        self.assertEqual(subtract(-1, 1), -2)
        self.assertEqual(subtract(-1, -1), 0)

    def test_multiply(self):
        self.assertEqual(multiply(2, 3), 6)
        self.assertEqual(multiply(-1, 1), -1)
        self.assertEqual(multiply(-1, -1), 1)

    def test_divide(self):
        self.assertEqual(divide(6, 3), 2)
        self.assertEqual(divide(-1, 1), -1)
        self.assertEqual(divide(-1, -1), 1)
        with self.assertRaises(ValueError):
            divide(1, 0)

if __name__ == '__main__':
    unittest.main(argv=[''], verbosity=2, exit=False)

```

6.1. Discuss with ChatGPT or a similar LLM all problems you encounter or code that you do not understand.

6.2. Write a small summary on the difference between modules and packages as you understand it . you can ask LLMs but need to provide your own thoughts on this

7. Lab Submission

7.1. Provide your feedback on this lab: what was comfortable and relatively easy, what was not.

Task 7. Submit all your source code and output ideally as a single PDF or two not 20. Attach all Python code you used or created for this lab. Print Google Colab Notebook and attach as a separate PDF with only all google colab code to Canvas.